Blockchain Lab Exp 3

Name: Rohan Lalchandani **Class:** D20A
Roll no: 31 **Batch:** A

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Challenges in Peer-to-Peer (P2P) Networks

Peer-to-Peer (P2P) networks are decentralized systems in which each node (peer) acts both as a client and a server. Instead of relying on a central authority, peers communicate directly with each other to share resources and information. Although P2P networks provide advantages such as decentralization, scalability, and fault tolerance, they also face several technical and security challenges.

1. Security Issues

One of the major challenges in P2P networks is maintaining security. Since there is no central authority controlling the network, malicious nodes may attempt to attack the system. Common security threats include **Sybil attacks**, where an attacker creates multiple fake identities, and **eclipse attacks**, where a node is surrounded by malicious peers that control the information it receives. These attacks can disrupt communication and compromise the integrity of the network.

2. Trust Management

In decentralized systems, nodes do not inherently trust each other. Establishing trust between unknown participants becomes difficult because peers may behave dishonestly. For example, some nodes may send incorrect data, refuse to forward messages, or attempt to manipulate transactions. Mechanisms such as cryptographic verification and consensus algorithms are used to maintain trust in such environments.

3. Network Scalability

As the number of nodes increases in a P2P network, maintaining efficient communication becomes challenging. Each node must maintain connections with multiple peers, which increases network overhead and bandwidth consumption. Handling a large volume of transactions and messages while maintaining performance is a major scalability concern.

4. Data Consistency

Ensuring that all nodes maintain the same version of data is another challenge. Because peers operate independently and may join or leave the network at any time, it becomes difficult to synchronize the data across all nodes. In blockchain systems, consensus mechanisms such as Proof of Work or Proof of Stake help maintain consistency across the network.

5. Resource Consumption

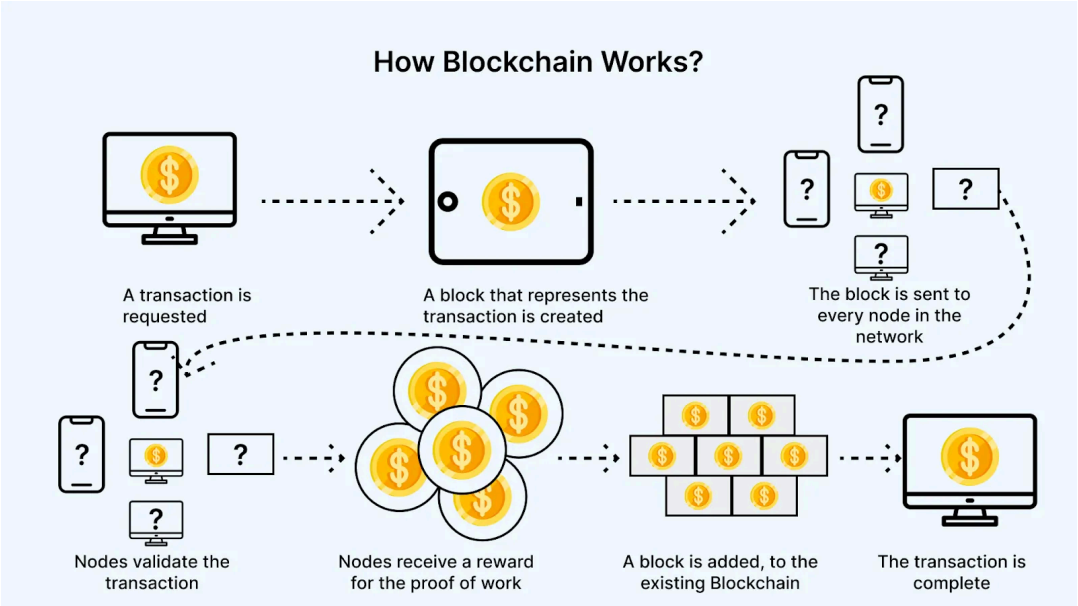
P2P networks can require significant computational resources, including processing power, memory, and bandwidth. Nodes that participate in activities like transaction validation or mining must handle large amounts of data, which may limit participation for devices with lower capabilities.

6. Network Latency

Communication delays can occur when transactions or messages are propagated across multiple nodes. High latency may slow down transaction confirmation times and reduce the efficiency of the network.

2. How Transactions Are Performed on the Network

In blockchain-based P2P networks, transactions represent the transfer of digital assets or data between users. The process of performing a transaction involves several steps to ensure security, transparency, and validity.



Transaction Creation

A transaction begins when a user initiates an action, such as transferring cryptocurrency to another user. The sender specifies the recipient's address, the amount to be transferred, and sometimes additional metadata. The transaction is then digitally signed using the sender's **private key** to ensure authenticity and prevent unauthorized modifications.

Transaction Broadcasting

Once created and signed, the transaction is broadcast to the peer-to-peer network. The node that receives the transaction forwards it to other connected peers. This propagation continues until the transaction reaches most nodes in the network.

Transaction Verification

Each node independently verifies the transaction before accepting it. Verification typically includes checking the digital signature, confirming that the sender has sufficient balance, and ensuring the transaction format follows protocol rules. Invalid transactions are rejected.

Inclusion in a Block

After verification, the transaction enters a pool of pending transactions. Miners or validators select transactions from this pool and include them in a candidate block. The block is then processed through the network's consensus mechanism.

Block Confirmation

Once a block containing the transaction is validated and added to the blockchain, the transaction is considered confirmed. Additional blocks added after it increase the security and finality of the transaction.

3. Role of Mempools

A **mempool (memory pool)** is a temporary storage area used by blockchain nodes to hold valid but unconfirmed transactions before they are included in a block.

Storage of Pending Transactions

When a transaction is broadcast to the network and verified by a node, it is placed into the node's mempool. The mempool acts as a waiting area for transactions until miners or validators select them for inclusion in a new block.

Transaction Prioritization

Transactions in the mempool are often prioritized based on transaction fees. Miners prefer transactions with higher fees because they receive those fees as rewards. Therefore, transactions offering higher fees are more likely to be included in the next block.

Network Synchronization

Each node maintains its own mempool, which may contain slightly different sets of transactions depending on network propagation timing. As blocks are mined, the transactions included in those blocks are removed from the mempool.

Prevention of Double Spending

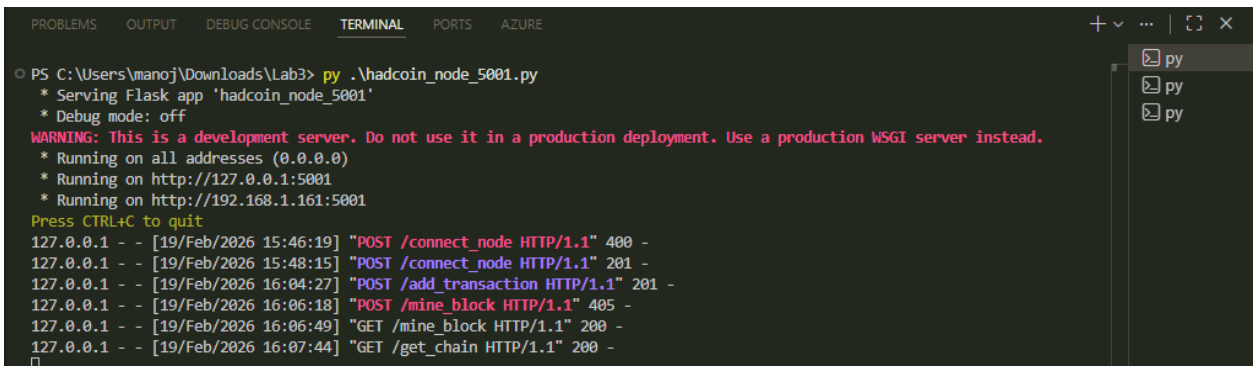
Before accepting a transaction into the mempool, nodes check whether the inputs of that transaction have already been spent in other pending transactions. This helps prevent double spending attempts.

Efficient Block Creation

The mempool allows miners to quickly assemble blocks by selecting transactions from a ready queue rather than waiting for new transactions to arrive.

Implementation:

1. Running hadcoin_node_5001, hadcoin_node_5002, hadcoin_node_5003 in 3 different terminals

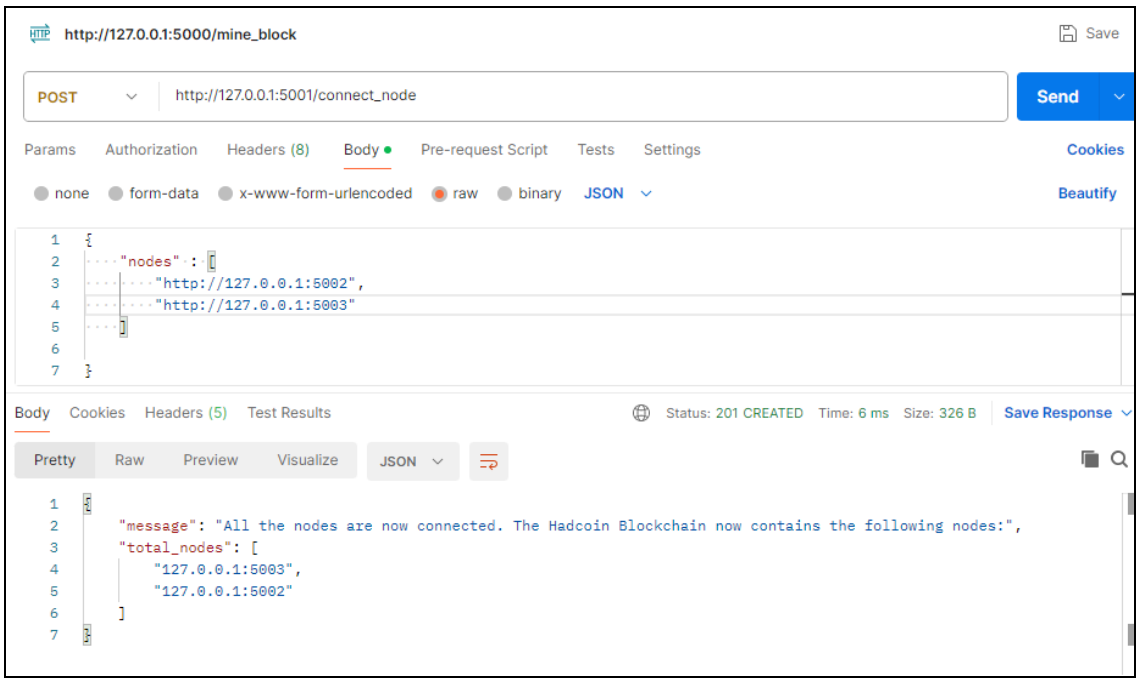


2. **On Node 5001 Postman Request:**

POST http://127.0.0.1:5001/connect_node

Body → raw → JSON:

```
{
 "nodes": [
  "http://127.0.0.1:5002",
  "http://127.0.0.1:5003"
 ]
}
```



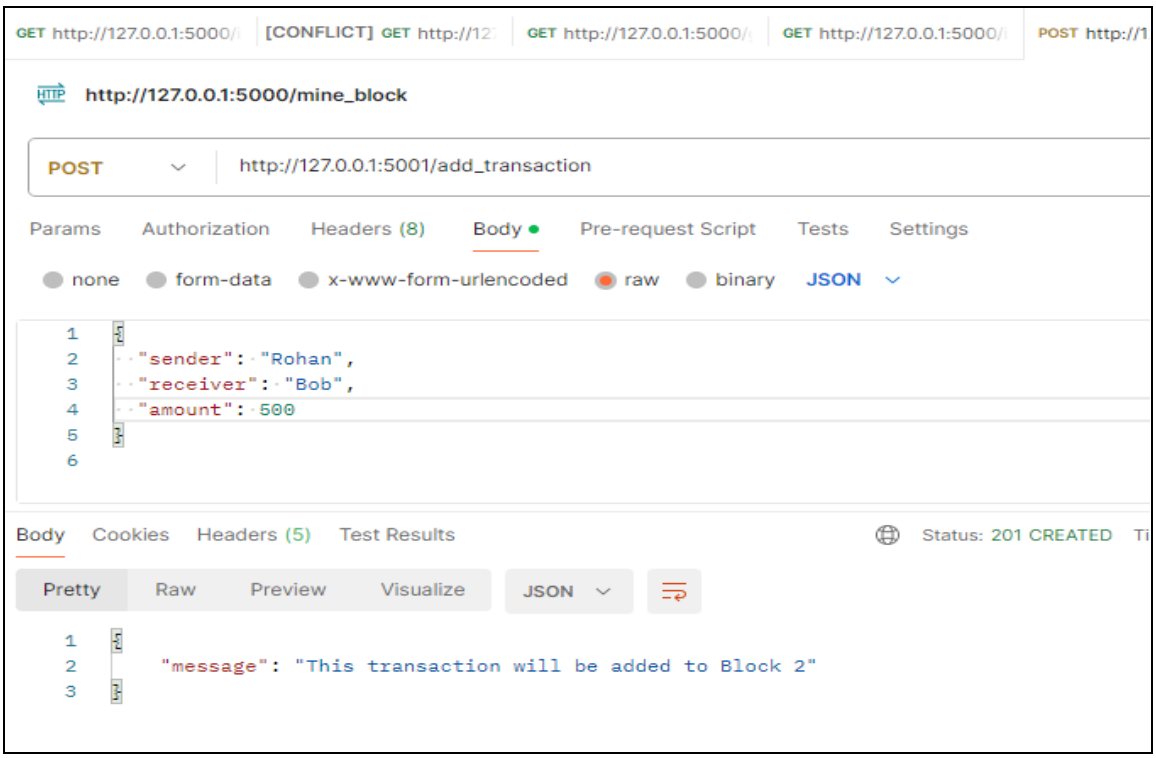
3. **Add Transactions (POST)**

Example on Node 5001:

POST http://127.0.0.1:5001/add_transaction

Body:

```
{
 "sender": "Rohan",
 "receiver": "Bob",
 "amount": 500
}
```

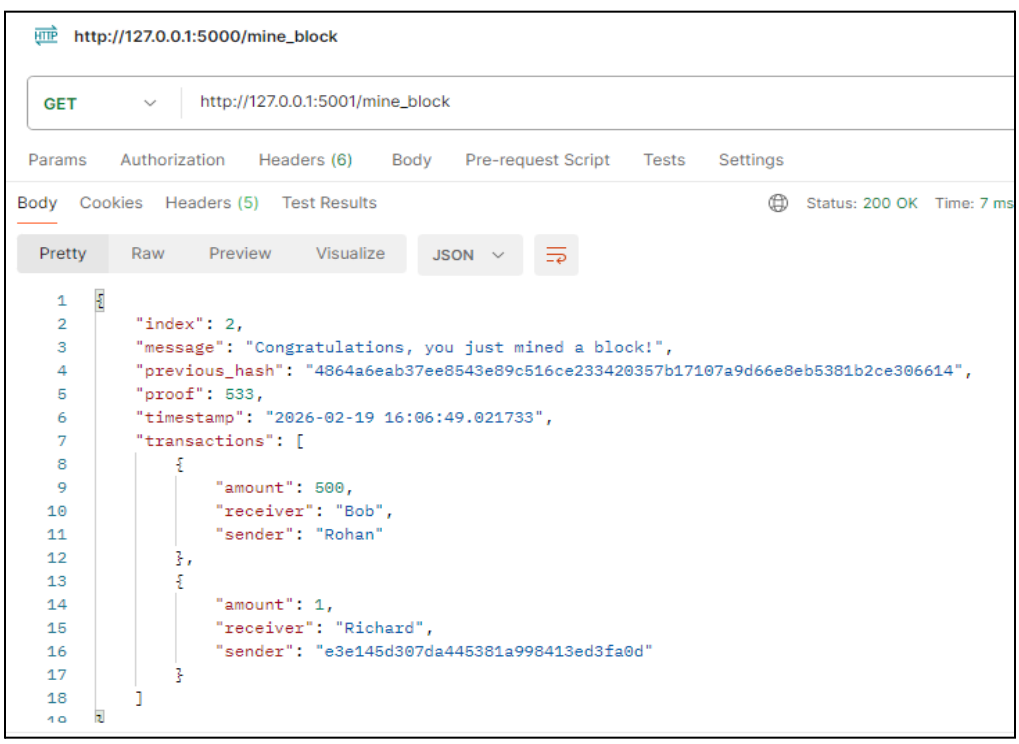


4. Mine Block (GET)

Mine from any node:

http://127.0.0.1:5001/mine_block

Block will include transactions, then transactions list becomes empty.



5. Fetch Blockchain (GET)

http://127.0.0.1:5000/mine_block

GET http://127.0.0.1:5001/get_chain

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results Status: 200 OK Time: 6 ms Size:

Pretty Raw Preview Visualize JSON


```
3        {
4            "index": 1,
5            "previous_hash": "0",
6            "proof": 1,
7            "timestamp": "2026-02-19 15:42:07.583125",
8            "transactions": []
9        },
10       {
11           "index": 2,
12           "previous_hash": "4864a6eab37ee8543e89c516ce233420357b17107a9d66e8eb5381b2ce306614",
13           "proof": 533,
14           "timestamp": "2026-02-19 16:06:49.021733",
15           "transactions": [
16               {
17                   "amount": 500,
18                   "receiver": "Bob",
19                   "sender": "Rohan"
20               },
```


6. Replace Chain (Consensus)

If one node has shorter chain:

http://127.0.0.1:5000/mine_block

GET http://127.0.0.1:5002/replace_chain

Params Authorization Headers (6) Body Pre-request Script Tests

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
1   {
2       "actual_chain": [
3           {
4               "index": 1,
5               "previous_hash": "0",
6               "proof": 1,
7               "timestamp": "2026-02-19 15:42:16.697499",
8               "transactions": []
9           }
10      ],
11      "message": "All good. The chain is the largest one."
12  }
```

Conclusion:

In this experiment, a basic cryptocurrency system was designed and implemented using Python by applying fundamental blockchain principles such as block generation, cryptographic hashing, Proof-of-Work mining, transaction processing, and decentralized networking. A Flask-based framework was used to enable communication between multiple nodes, with each node maintaining its own copy of the blockchain ledger. The mining process required solving the Proof-of-Work challenge, after which new blocks were added to the chain and miners received rewards.

The system also supported transaction validation and inclusion of verified transactions into newly mined blocks. Through this implementation, the experiment offered hands-on insight into the functioning of blockchain technology, demonstrating how consensus, decentralization, and mining together enable the secure operation of cryptocurrencies in distributed environments.